

DENEY RAPORU #6: UNIX'te Mesaj Kuyrukları (Message Queues)

Ad: Bartu

Soyad: Şanlı

Numara: 23220030065

Video Linki (En altta da mevcuttur) :

<https://drive.google.com/file/d/1b7T35o9qBhmaNXXleEvg2VDyuR2fQxKH/view?usp=sharing>

1. Giriş

Bu deneyin amacı, UNIX İşletim Sistemleri'nde süreçler arası iletişim (IPC - Inter Process Communication) mekanizmalarından biri olan Mesaj Kuyruklarını anlamak ve bu mekanizmayı kullanarak iki farklı sürecin (Gönderici ve Alıcı) haberleşmesini C programlama dili ile gerçekleştirmektir¹¹¹.

1.1 Mesaj Kuyrukları (Message Queues) Hakkında Temel Bilgiler

- Bir mesaj kuyruğu, bir süreç tarafından işletim sistemi çekirdeğinde yaratılır ve diğer süreçlerin erişimi için tutulur².
- Her mesaj kuyruğu bir anahtar (key) ile tanımlanır ve bu anahtar, iletişim kuracak tüm süreçler tarafından bilinmek zorundadır³.
- Kuyruktaki her mesajın pozitif bir tamsayı olan bir tipi ve bu tipe eşleşmiş bir verisi vardır⁴.
- Mesaj kuyrukları, işlev olarak pipe ile ortak bellek arasındadır; pipe'lardan daha esnek erişim sağlar fakat ortak bellek gibi gelişigüzel erişime izin vermez⁵.
- Kuyruk, msgctl() sistem çağırısı veya ipcrm kabuk komutu ile sistemden kaldırılabilir⁶.

2. Deneyin Gerçekleştirilmesi ve Adımlar

2.1 Adım 1 & 2: Temel Gönderici ve Alıcı Kodları

Bu adımda, tek bir mesaj gönderip alan ve hemen silen temel IPC yapısı kurulmuştur. Orijinal kodlardaki sözdizimi hataları giderilmiştir.

2.1.1 Gönderici (sndmsg.c)

```

GNU nano 7.2                                sndmsg.c
/* dosya adı : sndmsg.c */
/* Kullanım: program_adı anahtar tip boşluksuz_bir_mesaj */

#include <stdio.h>
#include <stdlib.h> // exit için
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h> // strcpy için

// msgbuf veri yapısı (types.h'de tanımlı değilse gereklidir)
struct msgbuf {
    long mtype;
    char mtext [100];
};

int main (int argc, char * argv[])
{
    int msid, v;
    struct msgbuf mess;

    if (argc != 4) {
        printf ("Kullanım: <key> <tip> <text>\n");
        exit (1);
    }

    // mesaj kuyruğunu yaratma veya erişim sağlama
    // IPC_CREAT: Kuyruk yoksa yarat, 0666: Okuma/Yazma izinleri
    msid = msgget ((key_t) atoi (argv[1]), IPC_CREAT | 0666);
    if (msid == -1) {
        printf("Mesaj kuyruğunu elde edemedi\n");
        exit (1);
    }

    // Komut satırından mesajı hazırlayalım
    mess.mtype = atoi (argv[2]); /* mesaj tipi */
    strcpy (mess.mtext, argv[3]); /* mesajın metni */

```

```

    // Komut satırından mesajı hazırlayalım
    mess.mtype = atoi (argv[2]); /* mesaj tipi */
    strcpy (mess.mtext, argv[3]); /* mesajın metni */

    // mesajı kuyruğa gönder
    // strlen(argv[3]) + 1: Metin uzunluğu + null sonlandırma karakteri
    v = msgsnd (msid, &mess, strlen(argv[3]) + 1, 0);

    if (v < 0) {
        printf("HATA : Mesaj kuyruğuna yazılamadı\n");
    }

    printf ("Gönderici sonlandı\n");
    return 0; // exit(0) yerine return 0 kullanıldı
}

```

2.1.2 Alıcı (rcvmsg.c)

```
/* dosya adı : rcvmsg.c */
/* Kullanım: program_adı anahtar tip */

#include <stdio.h>
#include <stdlib.h> // exit için
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <errno.h>

struct msgbuf {
    long mtype;
    char mtext [100];
};

int main (int argc, char * argv[])
{
    int msid, v;
    struct msgbuf mess;

    if (argc != 3) {
        printf ("Kullanım: <key> <tip>\n");
        exit (1);
    }

    // mesaj kuyruğu handle'ını al (0: Sadece erişim, yaratma yok)
    msid = msgget ((key_t) atoi (argv[1]), 0);
    if (msid == -1) {
        printf ("Bu key ile mesaj kuyruğuna erişim sağlanamadı\n");
        exit (1);
    }

    // kuyruktan bir mesaj al, mesaj kuyruktan silinmiş olur
    // 100: max mesaj boyutu, atoi(argv[2]): istenen mesaj tipi
    // IPC_NOWAIT: Mesaj yoksa bekleme, hemen hata dön
    v = msgrcv(msid, (struct msgbuf *)&mess, 100, atoi(argv[2]), IPC_NOWAIT);
```

```

// kuyruktan bir mesaj al, mesaj kuyruktan silinmiş olur
// 100: max mesaj boyutu, atoi(argv[2]): istenen mesaj tipi
// IPC_NOWAIT: Mesaj yoksa bekleme, hemen hata dön
v = msgrcv(msid, (struct msgbuf *)&mess, 100, atoi(argv[2]), IPC_NOWAIT);

if (v < 0) {
    if (errno == ENOMSG) {
        printf("Belirtilen tipte mesaj kuyrukta yok\n");
    } else {
        printf("HATA: kuyruktan okunamadı\n");
    }
} else {
    printf("[%ld] %s\n", mess.mtype, mess.mtext);
}

// mesaj kuyruğunu sistemden sil
if (msgctl(msid, IPC_RMID, 0) < 0) {
    printf("Hata: mesaj kuyruğu sistemden silinemedi\n");
    exit(1);
} else {
    printf("Mesaj kuyruğu (key = %ld ) sistemden silindi\n", (long)atoi(argv[2]));
}

return 0;
}

```

2.2 Adım 3 & 4: Çalıştırma ve Sonuç Analizi

Gönderici birden çok kez farklı anahtarlar ve tiplerle çalıştırılmış, ardından alıcı çalıştırılmıştır.

Key	Tip	Mesaj
1234	1	"Merhaba"
1234	2	"Nasılsın"
5678	1	"Deneme"

```

bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 1234 1 "Merhaba"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 1234 2 "Nasılsın"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 5678 1 "Deneme"
Gönderici sonlandı

```



```

bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid        owner          perms          used-bytes   messages
0x0000004d2  0            bartu_sanl     666            19           2
0x0000162e   1            bartu_sanl     666            7            1

```

```

bartu_sanli@bartu-computer:~/lab6$ ./rcvmsg 1234 1
[1] Merhaba
Mesaj kuyruğu (key = 1234 ) sistemden silindi
bartu_sanli@bartu-computer:~/lab6$ ./rcvmsg 1234 2
Bu key ile mesaj kuyruğuna erişim sağlanamadı

```

```

bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 1234 1 "2. Kez Merhaba"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 1234 3 "2. Kez Nasılsın"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./rcvmsg 1234 3
[3] 2. Kez Nasılsın
Mesaj kuyruğu (key = 1234 ) sistemden silindi
bartu_sanli@bartu-computer:~/lab6$ ./rcvmsg 1234 1
Bu key ile mesaj kuyruğuna erişim sağlanamadı

```

```

bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 1234 1 "deneme 3"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 2345 1 "deneme 4"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 3456 1 "deneme 5"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 4567 1 "deneme 6"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 5678 1 "deneme 7"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 6789 1 "deneme 8"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ./sndmsg 7890 1 "deneme 9"
Gönderici sonlandı
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

```

```

----- Message Queues -----
key          msqid        owner          perms          used-bytes   messages
0x0000004d2  3            bartu_sanl     666            9            1
0x000000929  4            bartu_sanl     666            9            1
0x000000d80  5            bartu_sanl     666            9            1
0x000011d7   6            bartu_sanl     666            9            1
0x0000162e   7            bartu_sanl     666            9            1
0x00001a85   8            bartu_sanl     666            9            1
0x00001ed2   9            bartu_sanl     666            9            1

```

Gözlem ve Analiz:

1. Gönderici, aynı key (1234) ile farklı tiplerde (1 ve 2) mesajlar gönderebilir. Her iki mesaj da aynı kuyrukta bekler.
2. Alıcı (./rcvmsg 1234 1) çalıştırıldığında, Tip 1 mesajını alır. Ancak, rcvmsg.c kodu mesajı aldıktan hemen sonra kuyruğu silmek için msgctl(msid, IPC_RMID, 0) komutunu içerdiği için, Tip 2 mesajı kuyrukla birlikte sistemden silinmiştir.
3. Alıcının kuyruğu silmesi, süreçler arası iletişimin sonlandırıldığını ve kaynakların temizlendiğini gösterir.

2.3 Adım 5: Sürekli ve Paralel İletişim

Bu adımda, süreçlerin paralel çalıştığı ve periyodik olarak haberleştiği bir senaryo (sndloop.c ve rcvloop.c) gerçekleştirilmiştir.

2.3.1 Gönderici (sndloop.c)

- sleep(5) ile 5 saniyede bir mesaj gönderilmiştir.
- Toplam 20 mesaj gönderildikten sonra sonlanmıştır.

```
/* dosya adı : sndloop.c */
/* Kullanım: program_adı anahtar tip boşluksuz_bir_mesaj */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
#include <unistd.h> // sleep için

#define MAX_MESAJ 20
#define BEKLEME_SURESI 5 // Saniye

struct msgbuf {
    long mtype;
    char mtext [100];
};

int main (int argc, char * argv[])
{
    int msid, v;
    struct msgbuf mess;
    int gonderilen_mesaj_sayisi = 0;

    // argüman kontrolü (key, tip, text)
    if (argc != 4) {
        printf ("Kullanım: <key> <tip> <text>\n");
        exit (1);
    }

    // Mesaj kuyruğunu yaratma veya erişim sağlama
    msid = msgget ((key_t) atoi (argv[1]), IPC_CREAT | 0666);
    if (msid == -1) {
        printf("HATA: Mesaj kuyruğunu elde edemedi\n");
        exit (1);
    }
}
```

```
GNU nano 7.2                                sndloop.c *
// Mesaj tipini ve metnini hazırla
mess.mtype = atoi (argv[2]);
char* base_text = argv[3];
char tam_mesaj[150]; // Mesaj metni + sayaç için yeterli alan

printf("Gönderici başladı. Anahtar: %s, Tip: %s. %d mesaj gönderecek.\n");

// 20 mesajı kuyruğa atacak döngü
while (gonderilen_mesaj_sayisi < MAX_MESAJ) {
    gonderilen_mesaj_sayisi++;

    // Mesaj metnini güncelleyelim (sayaç ekleyelim)
    sprintf(tam_mesaj, "%s #%d", base_text, gonderilen_mesaj_sayisi);
    strcpy(mess.mtext, tam_mesaj);

    // Mesajı kuyruğa gönder
    // strlen(tam_mesaj) + 1: Metin uzunluğu + null sonlandırma karakteri
    v = msgsnd (msid, &mess, strlen(tam_mesaj) + 1, 0);

    if (v < 0) {
        perror("HATA: Mesaj kuyruğuna yazılamadı");
        break; // Hata durumunda döngüden çık
    }

    printf("[%ld] Mesaj %d/%d gönderildi: %s\n", mess.mtype, gonderilen_mesaj_sayisi, MAX_MESAJ, tam_mesaj);

    // 5 saniye bekle
    if (gonderilen_mesaj_sayisi < MAX_MESAJ) {
        sleep(BEKLEME_SURESI);
    }
}

printf("Gönderici sonlandı. Toplam %d mesaj gönderildi.\n", gonderilen_mesaj_sayisi);
return 0;
}
```

2.3.2 Alıcı (rcvloop.c)

- msgrcv içinde tip=0 kullanılarak kuyruktaki sıradaki mesajlar tipine bakılmaksızın alınmıştır.
- IPC_NOWAIT bayrağı ile kuyrukta mesaj kalmayınca (errno == ENOMSG) döngüden çıkılmış ve kuyruk silinmiştir.


```
/* dosya adı : rcvloop.c */
/* Kullanım: program_adı anahtar */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <errno.h>
#include <unistd.h> // sleep için (isteğe bağlı, okuyucunun yavaşlamasını e>

struct msgbuf {
    long mtype;
    char mtext [100];
};

int main (int argc, char * argv[])
{
    int msid, v;
    struct msgbuf mess;
    long istenen_tip = 0; // msgrcv'de tip=0, kuyruktaki sıradaki mesajı al>
    int alinan_mesaj_sayisi = 0;

    // argüman kontrolü (sadece key)
    if (argc != 2) {
        printf ("Kullanım: <key>\n");
        exit (1);
    }

    // Mesaj kuyruğu handle'ını al (yaratma yok)
    msid = msgget ((key_t) atoi (argv[1]), 0);
    if (msid == -1) {
        printf ("Bu key (%s) ile mesaj kuyruğuna erişim sağlanamadı.\n", ar>
        exit (1);
    }

    printf("Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...\n");
```

```
GNU nano 7.2                                rcvloop.c *
printf("Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...\n");

// Kuyruktan tüm mesajları alacak döngü
while (1) {
    // msgrcv: 0 tipi ile sıradaki mesajı al. IPC_NOWAIT: mesaj yoksa b>
    v = msgrcv(msid, (struct msgbuf *)&mess, 100, istenen_tip, IPC_NOWAIT);

    if (v < 0) {
        // Hata kodu ENOMSG ise, kuyrukta bekleyen mesaj kalmamış demek>
        if (errno == ENOMSG) {
            printf("Belirtilen tipte/sıradaki mesaj kuyrukta yok. Tüm m>
            break; // Döngüden çık
        } else {
            perror ("HATA: Kuyruktan okunamadı");
            break; // Diğer hatalarda döngüden çık
        }
    }

    alinan_mesaj_sayisi++;
    printf ("Alındı (%d): [%ld] %s\n", alinan_mesaj_sayisi, mess.mtype,>

    // Okuma işleminin hızını yavaşlatmak (isteğe bağlı, gözlem için ya>
    // usleep(100000); // 100 milisaniye bekle
}

// Tüm mesajlar alınınca mesaj kuyruğunu sistemden sil
if (msgctl(msid, IPC_RMID, 0) < 0) {
    perror("Hata: Mesaj kuyruğu sistemden silinemedi");
    exit(1);
} else {
    printf("Mesaj kuyruğu (key = %s) sistemden silindi.\n", argv[1]);
}

printf("Alıcı sonlandı. Toplam %d mesaj alındı.\n", alinan_mesaj_sayisi>
return 0;
}
```

2.3.3 Paralel Çalıştırma ve Sonuç:

- Gönderici, ./sndloop 1234 1 "Veri" & komutu ile arka planda çalıştırılmıştır.
- Alıcı, ./rcvloop 1234 komutu ile hemen başlatılmıştır.
- **Sonuç:** Gönderici 5 saniyede bir mesajı kuyruğa atarken, Alıcı bu mesajları neredeyse anında kuyruktan çekmiş ve yazmıştır. Bu, mesaj kuyruklarının eşzamansız (asenكرون) haberleşme ve arabellekleme özelliğini göstermiştir.

```
bartu_sanli@bartu-computer:~/lab6$ ./sndloop 1234 1 "Loop Deneme Mesajı"
Gönderici başladı. Anahtar: 1234, Tip: 1. 20 mesaj gönderecek.
[1] Mesaj 1/20 gönderildi: Loop Deneme Mesajı #1
./rcv[1] Mesaj 2/20 gönderildi: Loop Deneme Mesajı #2
loop [1] Mesaj 3/20 gönderildi: Loop Deneme Mesajı #3
1234
[1] Mesaj 4/20 gönderildi: Loop Deneme Mesajı #4
[1] Mesaj 5/20 gönderildi: Loop Deneme Mesajı #5
[1] Mesaj 6/20 gönderildi: Loop Deneme Mesajı #6
[1] Mesaj 7/20 gönderildi: Loop Deneme Mesajı #7
[1] Mesaj 8/20 gönderildi: Loop Deneme Mesajı #8
[1] Mesaj 9/20 gönderildi: Loop Deneme Mesajı #9
[1] Mesaj 10/20 gönderildi: Loop Deneme Mesajı #10
[1] Mesaj 11/20 gönderildi: Loop Deneme Mesajı #11
[1] Mesaj 12/20 gönderildi: Loop Deneme Mesajı #12
[1] Mesaj 13/20 gönderildi: Loop Deneme Mesajı #13
[1] Mesaj 14/20 gönderildi: Loop Deneme Mesajı #14
[1] Mesaj 15/20 gönderildi: Loop Deneme Mesajı #15
[1] Mesaj 16/20 gönderildi: Loop Deneme Mesajı #16
[1] Mesaj 17/20 gönderildi: Loop Deneme Mesajı #17
[1] Mesaj 18/20 gönderildi: Loop Deneme Mesajı #18
[1] Mesaj 19/20 gönderildi: Loop Deneme Mesajı #19
[1] Mesaj 20/20 gönderildi: Loop Deneme Mesajı #20
Gönderici sonlandı. Toplam 20 mesaj gönderildi.
```

```
bartu_sanli@bartu-computer:~/lab6$ ./rcvloop 1234
Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...
Alındı (1): [1] deneme 3
Alındı (2): [1] Loop Deneme Mesajı #1
Alındı (3): [1] Loop Deneme Mesajı #2
Alındı (4): [1] Loop Deneme Mesajı #3
Alındı (5): [1] Loop Deneme Mesajı #4
Alındı (6): [1] Loop Deneme Mesajı #5
Alındı (7): [1] Loop Deneme Mesajı #6
Alındı (8): [1] Loop Deneme Mesajı #7
Alındı (9): [1] Loop Deneme Mesajı #8
Alındı (10): [1] Loop Deneme Mesajı #9
Alındı (11): [1] Loop Deneme Mesajı #10
Alındı (12): [1] Loop Deneme Mesajı #11
Alındı (13): [1] Loop Deneme Mesajı #12
Alındı (14): [1] Loop Deneme Mesajı #13
Alındı (15): [1] Loop Deneme Mesajı #14
Alındı (16): [1] Loop Deneme Mesajı #15
Alındı (17): [1] Loop Deneme Mesajı #16
Alındı (18): [1] Loop Deneme Mesajı #17
Alındı (19): [1] Loop Deneme Mesajı #18
Alındı (20): [1] Loop Deneme Mesajı #19
Alındı (21): [1] Loop Deneme Mesajı #20
Belirtilen tipte/sıradaki mesaj kuyrukta yok. Tüm mesajlar alındı.
Mesaj kuyruğu (key = 1234) sistemden silindi.
Alıcı sonlandı. Toplam 21 mesaj alındı.
```



```
bartu_sanli@bartu-computer:~/lab6$ ./sndloop 1234 1 "Loop Deneme Mesajı" &
[1] 1290
bartu_sanli@bartu-computer:~/lab6$ Gönderici başladı. Anahtar: 1234, Tip: 1.
20 mesaj gönderecek.
[1] Mesaj 1/20 gönderildi: Loop Deneme Mesajı #1
./rcvloop 1234[1] Mesaj 2/20 gönderildi: Loop Deneme Mesajı #2

Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...
Alındı (1): [1] Loop Deneme Mesajı #1
Alındı (2): [1] Loop Deneme Mesajı #2
Belirtilen tipte/sıradaki mesaj kuyruқта yok. Tüm mesajlar alındı.
Mesaj kuyruğı (key = 1234) sistemden silindi.
Alıcı sonlandı. Toplam 2 mesaj alındı.
bartu_sanli@bartu-computer:~/lab6$ ./rcvloop 1234
Bu key (1234) ile mesaj kuyruğına erişim sağlanamadı.
bartu_sanli@bartu-computer:~/lab6$ HATA: Mesaj kuyruğına yazılamadı: Invalid
argument
Gönderici sonlandı. Toplam 3 mesaj gönderildi.
./sndloop 1234 1 "Loop Deneme Mesajı" &
[2] 1293
[1] Done ./sndloop 1234 1 "Loop Deneme Mesajı"
bartu_sanli@bartu-computer:~/lab6$ Gönderici başladı. Anahtar: 1234, Tip: 1.
20 mesaj gönderecek.
[1] Mesaj 1/20 gönderildi: Loop Deneme Mesajı #1
./rcvloop 1234[1] Mesaj 2/20 gönderildi: Loop Deneme Mesajı #2
sndloop 1234 1 "Loop Deneme Mesajı" &[1] Mesaj 3/20 gönderildi: Loop Deneme
Mesajı #3
[1] Mesaj 4/20 gönderildi: Loop Deneme Mesajı #4
[1] Mesaj 5/20 gönderildi: Loop Deneme Mesajı #5
[1] Mesaj 6/20 gönderildi: Loop Deneme Mesajı #6
[1] Mesaj 7/20 gönderildi: Loop Deneme Mesajı #7
[1] Mesaj 8/20 gönderildi: Loop Deneme Mesajı #8
[1] Mesaj 9/20 gönderildi: Loop Deneme Mesajı #9
[1] Mesaj 10/20 gönderildi: Loop Denesndloop 1234 1 "Loop Deneme Mesajı" &[1
] Mesaj 11/20 gönderildi: Loop Deneme Mesajı #11
sndloop 1234 1 "Loop Deneme Mesajı"[1] Mesaj 12/20 gönderildi: Loop Deneme M
esajı #12
[1] Mesaj 13/20 gönderildi: Loop Deneme Mesajı #13
./rcvloop 1234
Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...
```

```

bartu_sanli@bartu-computer:~/lab6$ HATA: Mesaj kuyruğuna yazılamadı: Invalid
argument
Gönderici sonlandı. Toplam 14 mesaj gönderildi.
./sndloop & 1234 1 "Loop Deneme Mesajı"
[3] 1305
Kullanım: <key> <tip> <text>
1234: command not found
[2] Done ./sndloop 1234 1 "Loop Deneme Mesajı"
[3]+ Exit 1 ./sndloop
bartu_sanli@bartu-computer:~/lab6$ ./sndloop 1234 1 "Loop Deneme Mesajı" &
[1] 1316
bartu_sanli@bartu-computer:~/lab6$ Gönderici başladı. Anahtar: 1234, Tip: 1.
20 mesaj gönderecek.
[1] Mesaj 1/20 gönderildi: Loop Deneme Mesajı #1
[1] Mesaj 2/20 gönderildi: Loop Deneme Mesajı #2
[1] Mesaj 3/20 gönderildi: Loop Deneme Mesajı #3
./sndloop 1234 1 "Loop Deneme Mesajı" [1] Mesaj 4/20 gönderildi: Loop Deneme Me
sajı #4
[1] Mesaj 5/20 gönderildi: Loop Deneme Mesajı #5
[1] Mesaj 6/20 gönderildi: Loop Deneme Mesajı #6
.[1] Mesaj 7/20 gönderildi: Loop Deneme Mesajı #7
/rcvloop12[1] Mesaj 8/20 gönderildi: Loop Deneme Mesajı #8
34
-bash: ./rcvloop1234: No such file or directory
bartu_sanli@bartu-computer:~/lab6$ [1] Mesaj 9/20 gönderildi: Loop Deneme Me
sajı #9
./rcv1[1] Mesaj 10/20 gönderildi: Loop Deneme Mesajı #10
oop 1234
Alıcı başladı. Kuyruktan tüm mesajları (tip=0) alıyor...
Alındı (1): [1] Loop Deneme Mesajı #1
Alındı (2): [1] Loop Deneme Mesajı #2
Alındı (3): [1] Loop Deneme Mesajı #3
Alındı (4): [1] Loop Deneme Mesajı #4
Alındı (5): [1] Loop Deneme Mesajı #5
Alındı (6): [1] Loop Deneme Mesajı #6
Alındı (7): [1] Loop Deneme Mesajı #7
Alındı (8): [1] Loop Deneme Mesajı #8
Alındı (9): [1] Loop Deneme Mesajı #9
Alındı (10): [1] Loop Deneme Mesajı #10
Belirtilen tipte/sıradaki mesaj kuyrukta yok. Tüm mesajlar alındı.
Mesaj kuyruğu (key = 1234) sistemden silindi.

```

2.4 Adım 6: Yapısal Veri (Struct) Gönderme

Bu adımda, mesaj kuyruklarının sadece metin değil, bir C yapısını (struct) da taşıyabileceği gösterilmiştir.

2.4.1 Öğrenci Yapısı Tanımlaması:

C

```

typedef struct {
    int ogrenci_no;
    char ad[30];
    char sehir[30];
} tbilgi;

```


- Mesaj Tipleri: Tip 1 = tbilgi (Öğrenci), Tip 2 = Metin.

2.4.2 Gönderici (sndstruct.c)

- Tip 1 mesajlarında, tbilgi yapısı bellekteki haliyle msgsnd ile gönderilmiştir. (sizeof(tbilgi) mesaj boyutu olarak kullanılmıştır.)

```
GNU nano 7.2          sndstruct.c *
/* dosya adı : sndstruct.c */
/* Kullanım:
 * TIP 1 (Öğrenci): sndstruct <key> 1 <no> <ad> <sehir>
 * TIP 2 (Metin):   sndstruct <key> 2 <text>
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

// Öğrenci bilgilerini tutan yapı
typedef struct {
    int ogrenci_no;
    char ad[30];
    char sehir[30];
} tbilgi;

// Mesaj kuyruğu yapısı: mtext alanı en büyük veri yapısını (tbilgi) alacak
struct msgbuf {
    long mtype;
    char mtext [sizeof(tbilgi) + 1]; // Struct'ı tutmak için yeterli boyut
};

int main (int argc, char * argv[])
{
    int msid, v;
    struct msgbuf mess;
    long mesaj_tipi;
    size_t mesaj_boyutu;

    // Argüman kontrolü
    if (argc < 4 || argc > 6) {
        printf ("Hatalı Kullanım!\n");
        printf ("TIP 1 (Öğrenci): %s <key> 1 <no> <ad> <sehir>\n", argv[0]);
    }
}
```

```
printf ("TIP 2 (Metin):  %s <key> 2 <text>\n", argv[0]);
exit (1);
}

mesaj_tipi = atoi(argv[2]);

// 1. Mesaj kuyruğunu yaratma veya erişim sağlama
msid = msgget ((key_t) atoi (argv[1]), IPC_CREAT | 0666);
if (msid == -1) {
    perror("HATA: Mesaj kuyruğunu elde edemedi");
    exit (1);
}

mess.mtype = mesaj_tipi;

// 2. Mesaj tipine göre veriyi hazırla
if (mesaj_tipi == 1 && argc == 6) {
    // TIP 1: Öğrenci Bilgisi
    tbilgi ogrenci_bilgisi;

    ogrenci_bilgisi.ogrenci_no = atoi(argv[3]);
    strncpy(ogrenci_bilgisi.ad, argv[4], sizeof(ogrenci_bilgisi.ad) - 1);
    ogrenci_bilgisi.ad[sizeof(ogrenci_bilgisi.ad) - 1] = '\0';
    strncpy(ogrenci_bilgisi.sehir, argv[5], sizeof(ogrenci_bilgisi.sehir) - 1);
    ogrenci_bilgisi.sehir[sizeof(ogrenci_bilgisi.sehir) - 1] = '\0';

    // struct'ı mesaj tamponuna kopyala (char* tipine cast edilip)
    memcpy(mess.mtext, &ogrenci_bilgisi, sizeof(tbilgi));
    mesaj_boyutu = sizeof(tbilgi);

    printf("TIP 1 (Öğrenci) Mesajı Hazırlandı.\n");
} else if (mesaj_tipi == 2 && argc == 4) {
    // TIP 2: Metin Mesajı
    strncpy(mess.mtext, argv[3], sizeof(mess.mtext) - 1);
    mess.mtext[sizeof(mess.mtext) - 1] = '\0';
    mesaj_boyutu = strlen(mess.mtext) + 1; // Null sonlandırma ile meti
```

```

// TIP 2: Metin Mesajı
strncpy(mess.mtext, argv[3], sizeof(mess.mtext) - 1);
mess.mtext[sizeof(mess.mtext) - 1] = '\0';
mesaj_boyutu = strlen(mess.mtext) + 1; // Null sonlandırma ile meti
printf("TIP 2 (Metin) Mesajı Hazırlandı.\n");
} else {
printf ("HATA: Eksik veya yanlış argümanlar.\n");
exit (1);
}
// 3. Mesajı kuyruğa gönder
v = msgsnd (msid, &mess, mesaj_boyutu, 0);
if (v < 0) {
perror("HATA : Mesaj kuyruğuna yazılamadı");
} else {
printf ("%ld Mesaj başarıyla gönderildi. Boyut: %zu byte\n", mess
}
printf ("Gönderici sonlandı.\n");
return 0;
}
|

```

2.4.3 Alıcı (rcvstruct.c)

- Alınan mesajın tipi 1 ise, mesajın mtext alanındaki byte dizisi tekrar tbilgi yapısına dönüştürülmüş (casting/kopyalama) ve alanlarına erişilmiştir.

```
* dosya adı : rcvstruct.c */
* Kullanım: program_adı anahtar tip */

include <stdio.h>
include <stdlib.h>
include <sys/types.h>
include <sys/ipc.h>
include <sys/msg.h>
include <errno.h>
include <string.h>

/ Öğrenci bilgilerini tutan yapı
typedef struct {
    int ogrenci_no;
    char ad[30];
    char sehir[30];
    tbilgi;

/ Mesaj kuyruğu yapısı
struct msgbuf {
    long mtype;
    char mtext [sizeof(tbilgi) + 1]; // Struct'ı tutmak için yeterli boyut
};

nt main (int argc, char * argv[])

    int msid, v;
    struct msgbuf mess;
    long istenen_tip;

    if (argc != 3) {
        printf ("Kullanım: <key> <tip>\n");
        exit (1);
    }

    istenen_tip = atoi(argv[2]);
```



```
// 1. Mesaj kuyruğu handle'ını al
msid = msgget ((key_t) atoi (argv[1]), 0);
if (msid == -1) {
    printf ("Bu key ile mesaj kuyruğuna erişim sağlanamadı. (Belki gönd>
    exit (1);
}

// 2. Kuyruktan mesaj al
// msgrcv'deki boyut (sizeof(mess.mtext)), mesaj tamponunun alabileceği>
v = msgrcv(msid, (struct msgbuf *)&mess, sizeof(mess.mtext), istenen_ti>

if (v < 0) {
    if (errno == ENOMSG) {
        printf("Belirtilen tipte mesaj kuyrukta yok.\n");
    } else {
        perror ("HATA: Kuyruktan okunamadı");
    }
} else {
    printf("Mesaj alındı. Tip: [%ld], Boyut: %d byte\n", mess.mtype, v);

    // 3. Mesaj tipine göre içeriği yorumla
    if (mess.mtype == 1) {
        // TIP 1: Öğrenci Bilgisi
        if (v == sizeof(tbilgi)) {
            tbilgi alinan_bilgi;
            // Gelen veriyi struct'a kopyala
            memcpy(&alinan_bilgi, mess.mtext, sizeof(tbilgi));

            printf("--- Öğrenci Bilgisi ---\n");
            printf("No: %d\n", alinan_bilgi.ogrenci_no);
            printf("Ad: %s\n", alinan_bilgi.ad);
            printf("Şehir: %s\n", alinan_bilgi.sehir);
            printf("-----\n");
        } else {
            printf("HATA: Beklenen Öğrenci Yapısı Boyutu Eşleşmedi.\n")
        }
    }
}
```

```

        // Gelen veriyi struct'a kopyala
        memcpy(&alınan_bilgi, mess.mtext, sizeof(tbilgi));

        printf("--- Öğrenci Bilgisi ---\n");
        printf("No: %d\n", alınan_bilgi.ogrenci_no);
        printf("Ad: %s\n", alınan_bilgi.ad);
        printf("Şehir: %s\n", alınan_bilgi.sehir);
        printf("-----\n");
    } else {
        printf("HATA: Beklenen Öğrenci Yapısı Boyutu Eşleşmedi.\n");
    }

    } else if (mess.mtype == 2) {
        // TIP 2: Metin Mesajı
        printf("--- Metin Mesajı ---\n");
        printf("İçerik: %s\n", mess.mtext);
        printf("-----\n");
    } else {
        printf("Bilinmeyen Mesaj Tipi.\n");
    }
}

// 4. Mesaj kuyruğunu sistemden sil
if (msgctl(msid, IPC_RMID, 0) < 0) {
    perror("Hata: Mesaj kuyruğu sistemden silinemedi");
    exit(1);
} else {
    printf("Mesaj kuyruğu (key = %s) sistemden silindi.\n", argv[1]);
}

return 0;
}

```

2.5 Adım 7: Kaynak Yönetimi ve Temizleme

Bu adımda, sistemde kalan (orphaned) veya program tarafından silinmemiş mesaj kuyruklarının komut satırı ile yönetimi gösterilmiştir.

2.5.1 Kuyruk Kontrolü

Sistemdeki tüm mesaj kuyrukları kontrol edilir:

Ekran Görüntüsü 1: ipcs -q çıktısı

```

bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner      perms      used-bytes  messages
0x00000929   4          bartu_sanl 666        9           1
0x00000d80   5          bartu_sanl 666        9           1
0x000011d7   6          bartu_sanl 666        9           1
0x0000162e   7          bartu_sanl 666        9           1
0x00001a85   8          bartu_sanl 666        9           1
0x00001ed2   9          bartu_sanl 666        9           1

```

2.5.2 Kuyruk Silme

Belirli bir anahtar (key) veya ID (msqid) ile mesaj kuyruğu sistemden silinir⁸:

Bash

Örnek Silme: Key 1234 ile

ipcrm -Q 1234

VEYA

msqid 65536 ile (ipcs çıktısından)

ipcrm -q 65536

Ekran Görüntüsü 2: ipcrm komutunun uygulanması ve sonrası ipcs çıktısı

```
bartu_sanli@bartu-computer:~/lab6$ ipcrm
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00000929   4          bartu_sanl 666         9             1
0x00000d80   5          bartu_sanl 666         9             1
0x000011d7   6          bartu_sanl 666         9             1
0x0000162e   7          bartu_sanl 666         9             1
0x00001a85   8          bartu_sanl 666         9             1
0x00001ed2   9          bartu_sanl 666         9             1

bartu_sanli@bartu-computer:~/lab6$ ipcrm
bartu_sanli@bartu-computer:~/lab6$ ipcrm -q 4
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00000d80   5          bartu_sanl 666         9             1
0x000011d7   6          bartu_sanl 666         9             1
0x0000162e   7          bartu_sanl 666         9             1
0x00001a85   8          bartu_sanl 666         9             1
0x00001ed2   9          bartu_sanl 666         9             1

bartu_sanli@bartu-computer:~/lab6$ ipcrm -q 5
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x000011d7   6          bartu_sanl 666         9             1
0x0000162e   7          bartu_sanl 666         9             1
0x00001a85   8          bartu_sanl 666         9             1
0x00001ed2   9          bartu_sanl 666         9             1
```

```

bartu_sanli@bartu-computer:~/lab6$ ipcrm -q 5
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner        perms        used-bytes   messages
0x000011d7  6          bartu_sanl   666          9            1
0x0000162e  7          bartu_sanl   666          9            1
0x00001a85  8          bartu_sanl   666          9            1
0x00001ed2  9          bartu_sanl   666          9            1

bartu_sanli@bartu-computer:~/lab6$ ipcrm -Q 7890
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner        perms        used-bytes   messages
0x000011d7  6          bartu_sanl   666          9            1
0x0000162e  7          bartu_sanl   666          9            1
0x00001a85  8          bartu_sanl   666          9            1

bartu_sanli@bartu-computer:~/lab6$ ipc -Q 6789
Command 'ipc' not found, but there are 25 similar ones.
bartu_sanli@bartu-computer:~/lab6$ ipcrm -Q 6789
bartu_sanli@bartu-computer:~/lab6$ ipcrm -Q 5678
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner        perms        used-bytes   messages
0x000011d7  6          bartu_sanl   666          9            1

bartu_sanli@bartu-computer:~/lab6$ ipcrm -Q 4567
bartu_sanli@bartu-computer:~/lab6$ ipcs -q

----- Message Queues -----
key          msqid      owner        perms        used-bytes   messages

```

3. Sonuç

Bu deney, UNIX ortamında Mesaj Kuyruklarının temel çalışma prensiplerini, mesaj gönderme/alma sistem çağrılarını (msgget, msgsnd, msgrcv, msgctl), ve yapısal verinin IPC aracılığıyla nasıl taşınacağını göstermiştir. Ayrıca, paralel çalışan süreçler arası veri alışverişi simüle edilmiş ve sistem kaynaklarının yönetimi (ipcs, ipcrm) öğrenilmiştir.

4. Ekler

Video Kaydı Linki:

<https://drive.google.com/file/d/1b7T35o9qBhmaNXXleEvg2VDyuR2fQxKH/view?usp=sharing>